CSCE-553 Software Engineering

Milestone 5: Searching

Lagniappe Sign-Up

Summary

Milestone 5 focused on implementing search functionality into Langiappe Sign-Up via a PostgreSQL query and indexing. Whenever a user searches for a word or phrase on the application, the database checks the various indexes on every event object in the database and creates a score for how similar it is to the query the user has provided. An API was implemented in order to return each of these events.

On top of returning the similar objects to the user's query, the count of the various categories in the query are returned as well as a facet. A user can click on any of the categories on the page for their query and immediately get sent to the page with all of the objects from that query in that category.

On top of these, there were also upgrades done to search. Whenever a user begins to type out a query, the database will return names of the top 5 closest matching objects to that query on the same page without even having to press the search function that can be searched in a similar way to the category facet. It also checks if the query contains a typo.

A vector attribute was given to every event object in order to create a vector database. Each event object has its own vector that corresponds to the indexes inside of it. Similar words are closer together in vector space. The closest objects in this vector space are returned to the user based on similarity.

After these and the API were implemented, load tests were done on various queries that ranged in return size from 0 objects to 913 objects. These queries have a mix of only searching on a query and searching on a query and choosing a specific category. All of these were tested on a localhost, a Render deployment of the application, and an Azure deployment of the application.

The load test results showed that, even at low concurrency, there was usually a low RPS rate and a long response time. As the number of concurrent users increases, the error rate tends to increase dramatically and the max response time goes up exponentially on all deployment methods. On some deployments, the server used even crashed during load testing.

1. SQL Queries

In order to begin coding the search function in Django, what attributes the database would search through in order to do this searching would have to be evaluated.

The events in the database have various attributes, including its title, description, location, time, category, and organizer first and last name. The text in these attributes is inputted by the users that created them, the organizer of the event. The most important of these are the title, description, location, category, and name of the organizer in that order. These would be the attributes that would be checked in order to return a search functionality.

The relevancy of an attribute as it relates to the query helped determine the weights to be used for each attribute. It was determined that the title had the highest priority in order to set weights for the search view that was to be created. The title is the attribute of the event that is most likely to contain crucial information about the event as a whole, so returning events that have objects that similar titles to the query would be the most important. The description would also contain crucial information, but because event descriptions are usually a magnitude longer than the event title, the words in the description are still important but not as important than the title. As the relevancy of the attribute to a query decreased, so did the value of the weight. The order of weights in decreasing value is as follows: title, description, location, category, and name of the organizer in that order.

This helped determine how the sql functions for this search would be created.

```sql
drop materialized view if exists search.event_search;

create materialized view search.event_search as
select
  e."EventID" as "id",
  e."Title" as "title",
  e."Description" as "description",
  e."Location" as "location",
  e."DateTime" as "datetime",
  c."Name" as "category",
  u.first_name as "orgfirstname",
  u.last_name as "orglastname",
  setweight(to_tsvector('english', e."Title"), 'A') ||
  setweight(to_tsvector('english', coalesce(e."Description", '')), 'B') ||
  setweight(to_tsvector('english', e."Location"), 'C') ||
  setweight(to_tsvector('english', c."Name"), 'D') ||
  setweight(to_tsvector('english', coalesce(u.first_name, '') ||  ' ' || coalesce(u.last_name, '')), 'D') as fts
from events_event e
left join events_users u on u."UserID" = e."OrganizerID_id"
left join events_category c on c."CategoryID" = e."CategoryID_id";
```

Figure 1: Denormalized Search View search.event_search created Lagniappe Sign-Up

```
create unique index product_search_id
  on search.event_search (id);

create index event_search_id
  on search.event_search using gin (fts);
```

Figure 2: Index Creation

```
select cron.schedule(
  'refresh-event-search',
  '*/5 * * * *',
  $$refresh materialized view concurrently search.search.event_search
);
```

Figure 3: Cron refresh

```
create or replace function search.search_events(
  q text, cat test default null,
  lim int default 30 off int default 0)
returns table(id text, title text, description text, location text, datetime text, category text, firstname text, lastname text, rank real)
language sql stable as $$
  select s.id, s.title, s.description s.location s.datetime, s.category, s.firstname, s.lastname
    ts_rank(s.fts, websearch_to_tsquery('english', q))
  from search.event_search s
  where s.fts @@ websearch_to_tsquery('english', q)
  order by 6 desc, s.id
  limit lim offset off;
$$;
```

Figure 4: Search function for Lagniappe Sign-Up

2. Django Implementation

After these were created, these queries had to be implemented in some way in Django. The migration with the code was created, EventSearch was created as a model, and the search view and template were created.

Next, an API view had to be created for the search page. Lagniappe Sign-Up uses Django's built in Rest framework for its API implementation, so the implementation for this API view differs than the one seen in the slides.

3. Autocomplete

One of the objectives for this milestone was to implement autocomplete. As a user inserts text in the search box, there should be suggestions for what the user could be searching for on the same page.

This was achieved via creating a separate view that gets the top 5 best results what the user has typed out. A separate template for this view lists the title of these 5 closest rated events and hyperlinks them to a search for the title of that event. Inside of the template for the search view is a div that contains the template for the autocomplete view.

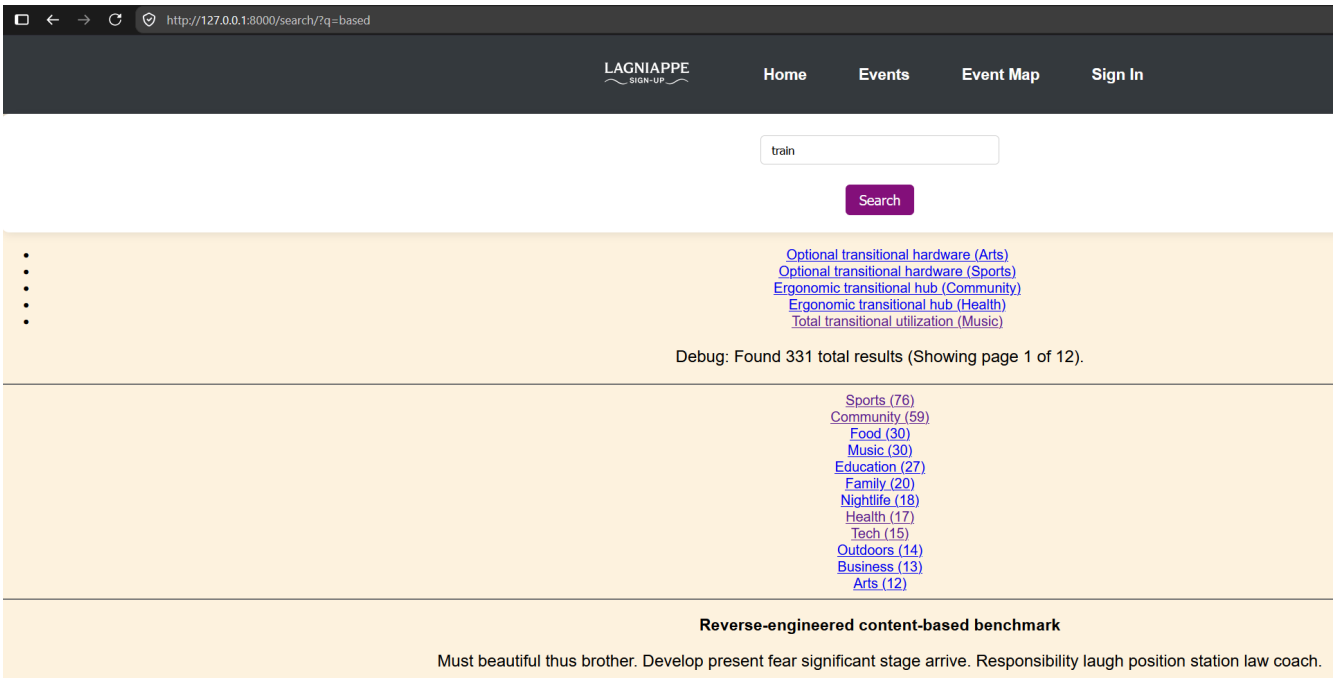


Figure 5: Implementation of Autocomplete into Lagniappe Sign-up. Notice how the url is for a search for the word "based" yet there are 5 results for "train," which is in the searchbox

4. Typo Tolerance

Another objective for this milestone is to implement typo tolerance. This is done through pg_trgm. This implements trigrams into the search query. The query a user creates is split into chunks

that consist of at most 3 letters. These chunks are compared to the chunks used in the rest of the database. Events that share chunks with the query are given a similarity value. The higher the similarity value event has, the more the event's chunks match with the query's chunks.

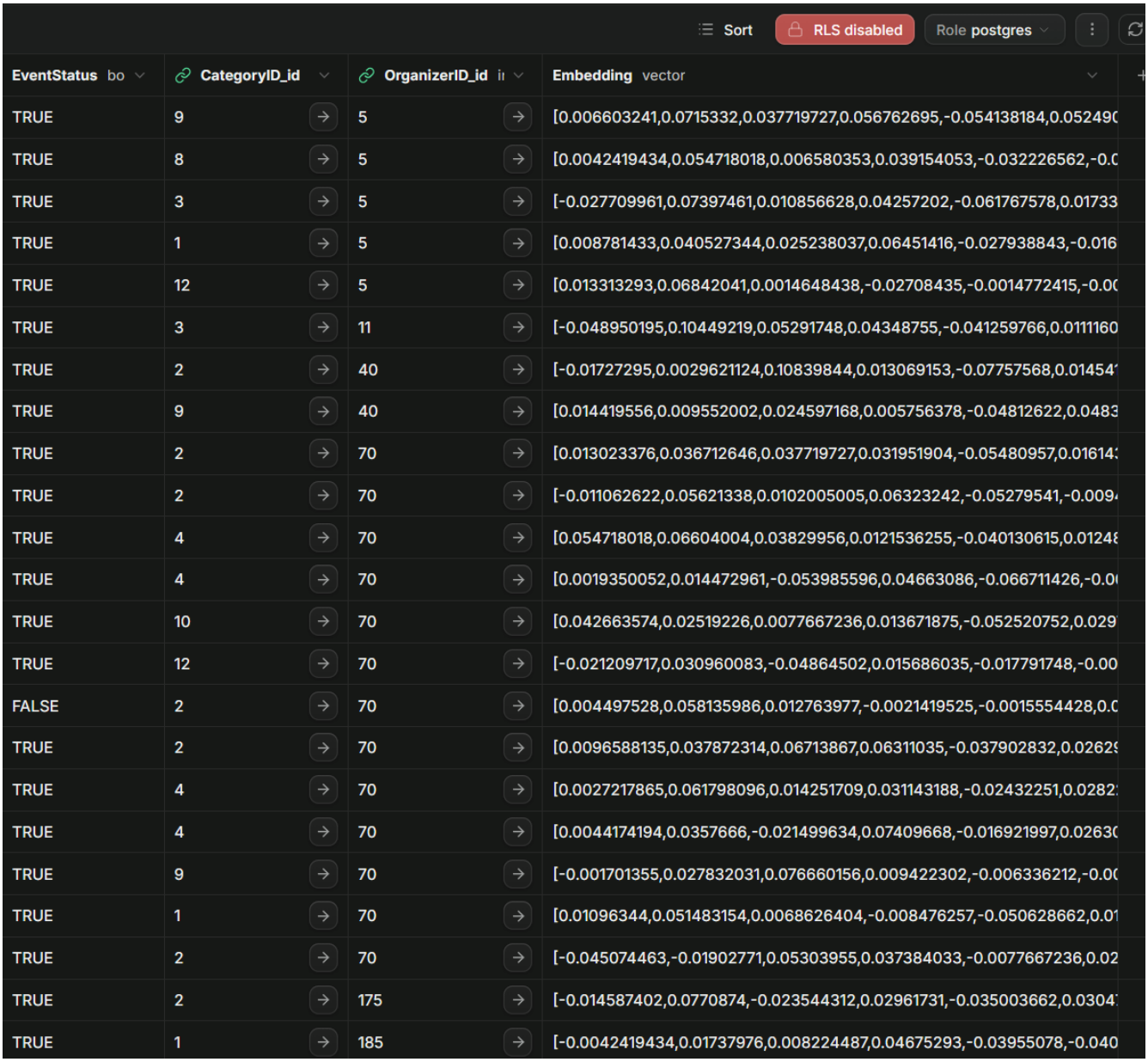
The way this implementation works is by getting all of the trigrams from the text of the events. From there, these trigrams are compared with the trigrams for the user query. If the similarity of the text of an event is higher than a specified amount, then it is included for the query. For this implementation, both the trigram similarity and a value associated with the vector associated with an event determine if an event will be included for a query. Read the next section to learn more about this vector rating.

5. Vector Databases

The final objective for this milestone was to implement a vector database into our application and to have our search queries implement this database to return similar objects in our queries.
The way a vector database works is via computing a numerical vector based on the words inside of it. The point that this vector creates is near the same point as other objects with similar names. The closer two objects are in vector space, the more similar they are.

These distances can be used in order to find two events that are similar to each other. Events with similar words in their title, description, etc, will be closer together in the vector database. We can set a certain distance that can be used to set a parameter that an event must meet in order to be included in the query.

First, the event model had to be updated to include the vectors at which they sit in vector space. It was determined that the vector would be 738 dimensions since this is slightly below the recommended dimension size for the small vector creation model that OpenAI uses.



EventStatus bo	CategoryID_id	OrganizerID_id	Embedding vector
TRUE	9	5	[0.006603241,0.0715332,0.037719727,0.056762695,-0.054138184,0.052490
TRUE	8	5	[0.0042419434,0.054718018,0.006580353,0.039154053,-0.032226562,-0.0
TRUE	3	5	[-0.027709961,0.07397461,0.010856628,0.04257202,-0.061767578,0.01733
TRUE	1	5	[0.008781433,0.040527344,0.025238037,0.06451416,-0.027938843,-0.016
TRUE	12	5	[0.013313293,0.06842041,0.0014648438,-0.02708435,-0.0014772415,-0.0(
TRUE	3	11	[-0.048950195,0.10449219,0.05291748,0.04348755,-0.041259766,0.0111160
TRUE	2	40	[-0.01727295,0.0029621124,0.10839844,0.013069153,-0.07757568,0.01454
TRUE	9	40	[0.014419556,0.009552002,0.024597168,0.005756378,-0.04812622,0.0483
TRUE	2	70	[0.013023376,0.036712646,0.037719727,0.031951904,-0.05480957,0.01614
TRUE	2	70	[-0.011062622,0.05621338,0.0102005005,0.06323242,-0.05279541,-0.009
TRUE	4	70	[0.054718018,0.06604004,0.03829956,0.0121536255,-0.040130615,0.0124
TRUE	4	70	[0.0019350052,0.014472961,-0.053985596,0.04663086,-0.066711426,-0.0(
TRUE	10	70	[0.042663574,0.02519226,0.0077667236,0.013671875,-0.052520752,0.029
TRUE	12	70	[-0.021209717,0.030960083,-0.04864502,0.015686035,-0.017791748,-0.00
FALSE	2	70	[0.004497528,0.058135986,0.012763977,-0.0021419525,-0.0015554428,0.0
TRUE	2	70	[0.0096588135,0.037872314,0.06713867,0.06311035,-0.037902832,0.0262
TRUE	4	70	[0.0027217865,0.061798096,0.014251709,0.031143188,-0.02432251,0.0282
TRUE	4	70	[0.0044174194,0.0357666,-0.021499634,0.07409668,-0.016921997,0.0263(
TRUE	9	70	[-0.001701355,0.027832031,0.076660156,0.009422302,-0.006336212,-0.0(
TRUE	1	70	[0.01096344,0.051483154,0.0068626404,-0.008476257,-0.050628662,0.01
TRUE	2	70	[-0.045074463,-0.01902771,0.05303955,0.037384033,-0.0077667236,0.02
TRUE	2	175	[-0.014587402,0.0770874,-0.023544312,0.02961731,-0.035003662,0.0304
TRUE	1	185	[-0.0042419434,0.01737976,0.008224487,0.04675293,-0.03955078,-0.040

Figure 6: Populated vector attribute for each event

From there, all it took was updating the search view create a vector for the user's query and only includes events that are similar to it. Cosine distance calculates distance between the query vector and an event object. If the distance is less than a specified distance, then it is included. After experimenting with the values for both cosine distance and trigram similarity, it was found that a cosine distance less than 0.3 and a text similarity of 0.15 produced the a good range of returned values that would help with load testing. A blank query would return nothing, and a query for the word "education" returned over 900 results. In a later section, you will see the queries that were used for the

load tests.

It should be noted that implementing trigram similarity and calculating cosine distance noticeably increased response times on the application. You can view Section 9 to see how the load tests on the application went.

6. API verification.

In order to verify that these queries did in fact return what they are supposed to, the contents in the HTTP body had to be checked. To do this, the API url for each of these queries were checked using Postman.

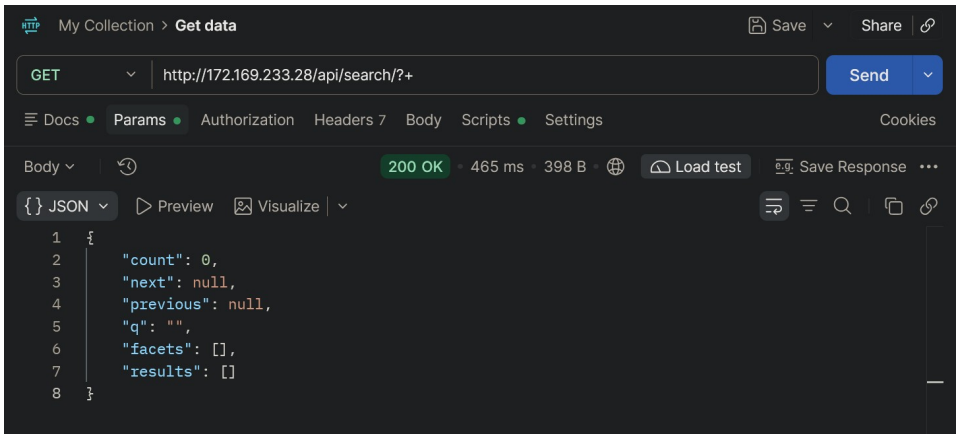


Figure 7: API check for blank query

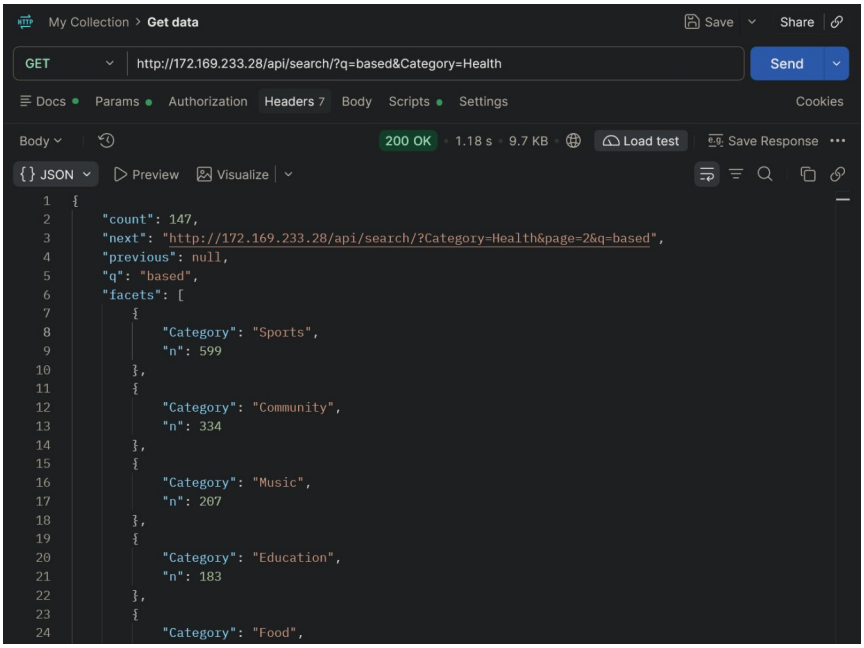


Figure 8: API check for “based” with the “Health” category query

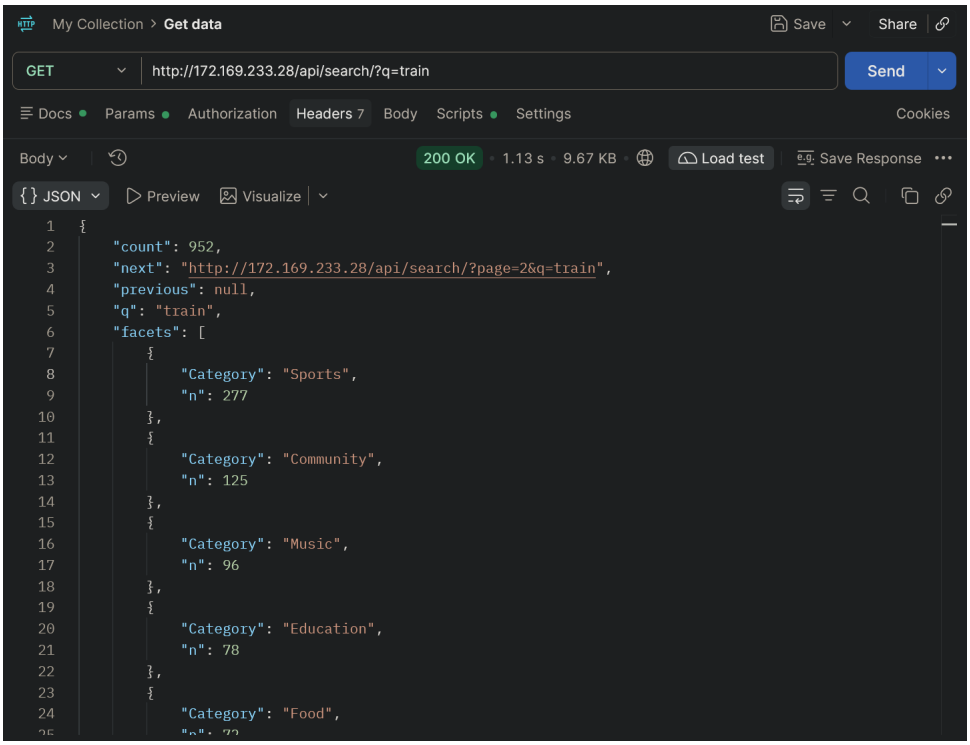


Figure 9: API check for “train” query

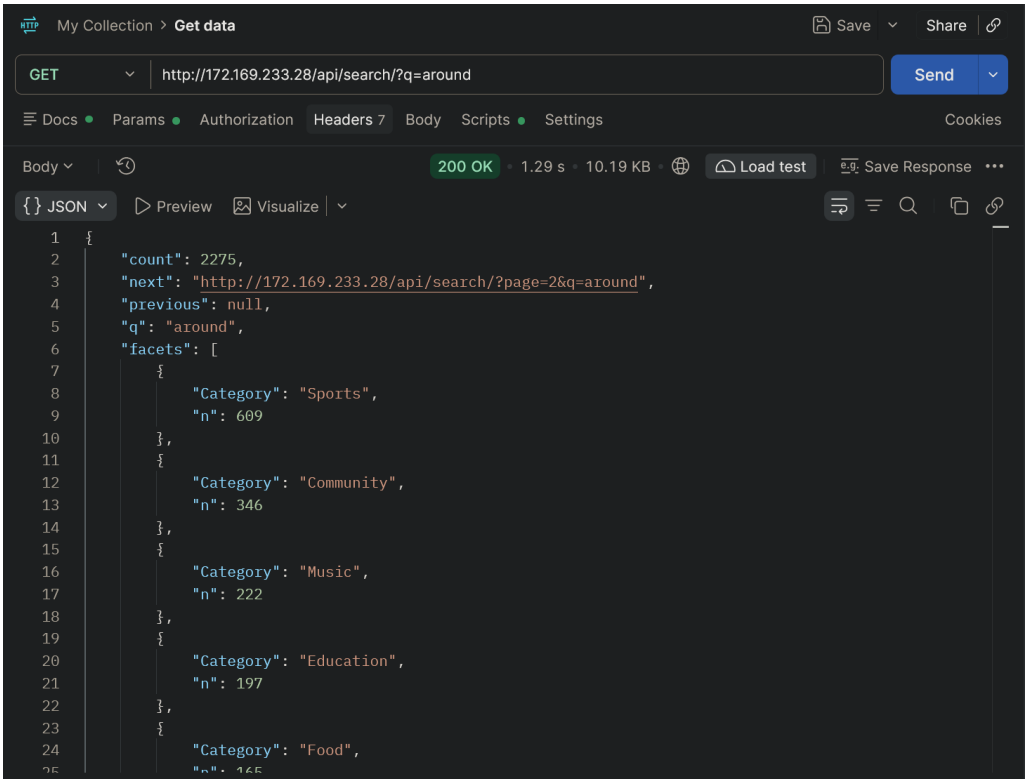


Figure 10: API check for “around” query

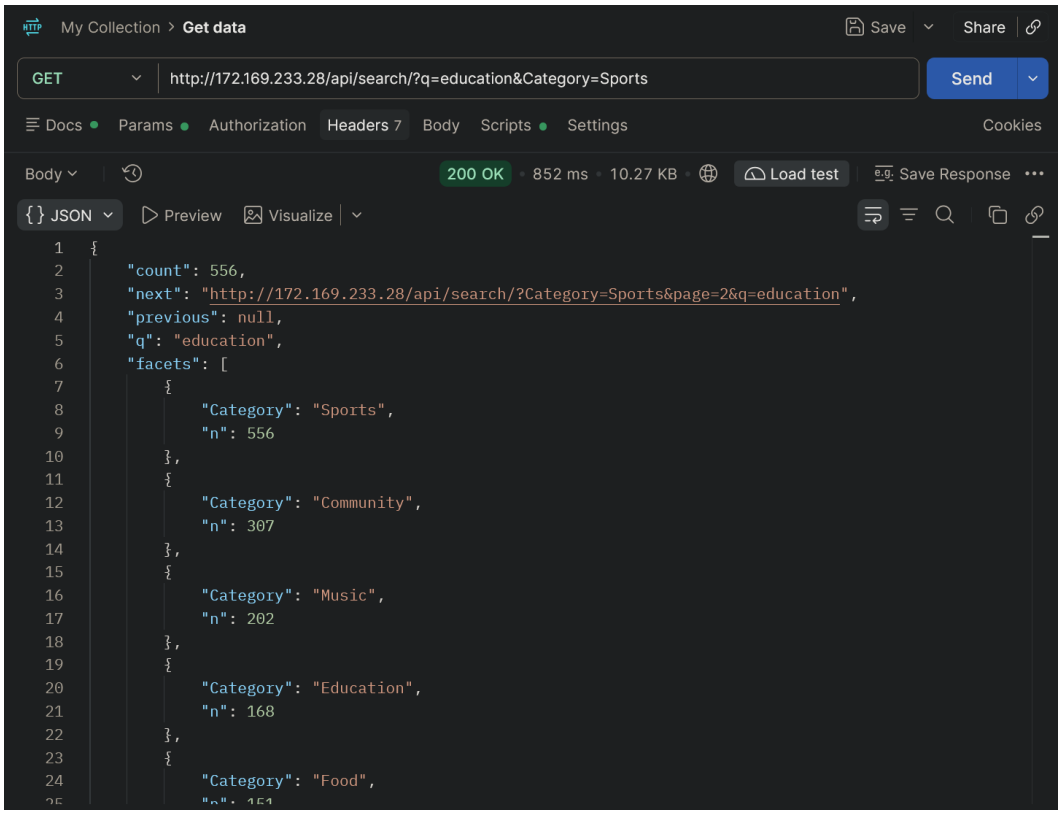


Figure 11: API check for “education” with the “Sports” category query

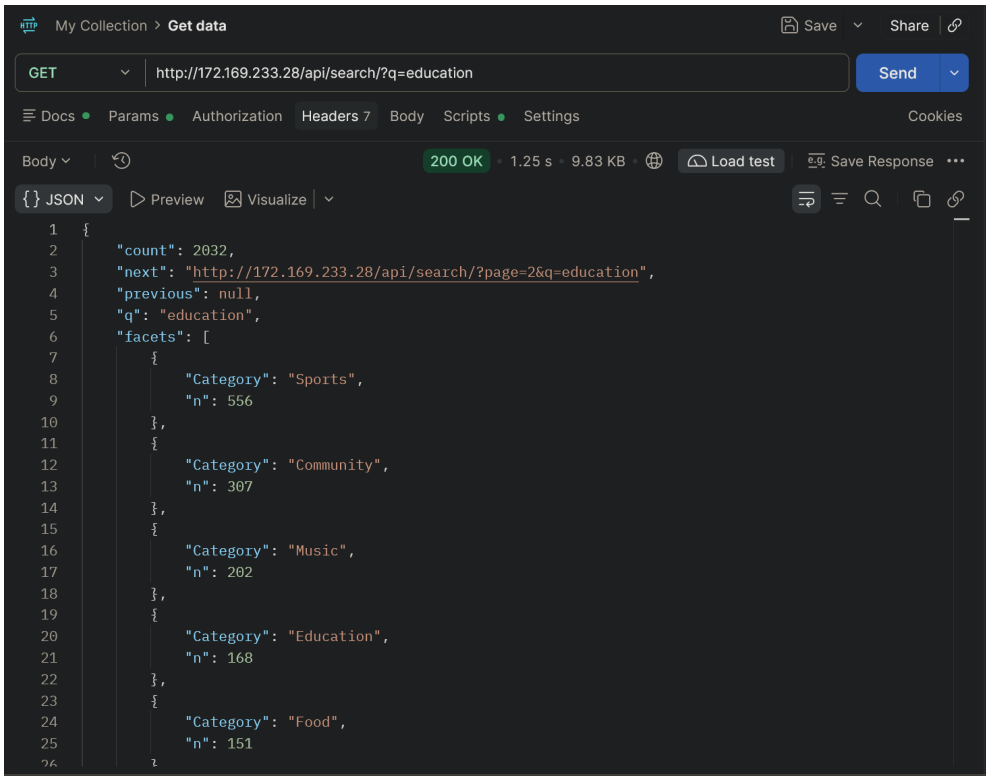


Figure 12: API check for “education” query

7. Render and Azure configuration

The load tests done on Lagniappe Sign-Up were done on locally hosted url, a Render deployment of the application, and an Azure deployment of the application in that order.

The Render server used to deploy the application used a paid tier of the service. The total memory available on this server is only 512 megabytes and is limited to only half of a CPU.

The Azure Virtual Machine used to load test this application had the following configuration:

- Resource Group: `InternetScale-Lab3`
- VM Name: `django-web-server`
- Region: Denmark East
- VM Size: `Standard_B2ats_v2`
- vCPU: 2
- RAM: 1 GiB
- Operating System: Ubuntu Server 24.04 LTS
- Architecture: x64
- OS Disk: Premium SSD P6, 64 GiB
- Allowed inbound ports:

- SSH 22
- HTTP 80

8. New Load Testing Script

The provided load_baseline.py file provided has been improved to be able to load test search queries. It was created as a new file, search_load_baseline.py, and modifying it from there. The new file was done in order to verify that the updated version of the script did not overwrite the code of the original script, but it was deemed worthy to keep them separate.

The path given to these load tests is the same throughout: /api/search/. The way queries are given for this script is by adding the parameter --query followed by the specified query in parenthesis in the command to run the script. Specifying a category works in a similar way. The parameter --category followed by the name of the category that will be searched in the load test command. For load tests that did not need to be run with a category specified, the argument for category was not used in the command.



Figure 13: Successful load test using the new search script. The chosen query is “based” and the category is “Health”

In order to verify that the script was in fact checking the correct URLs with this update, the terminal that launched the application on localhost, which outputs every single time a url is checked out. This verified that the new load test script was able to access the correct URLs.

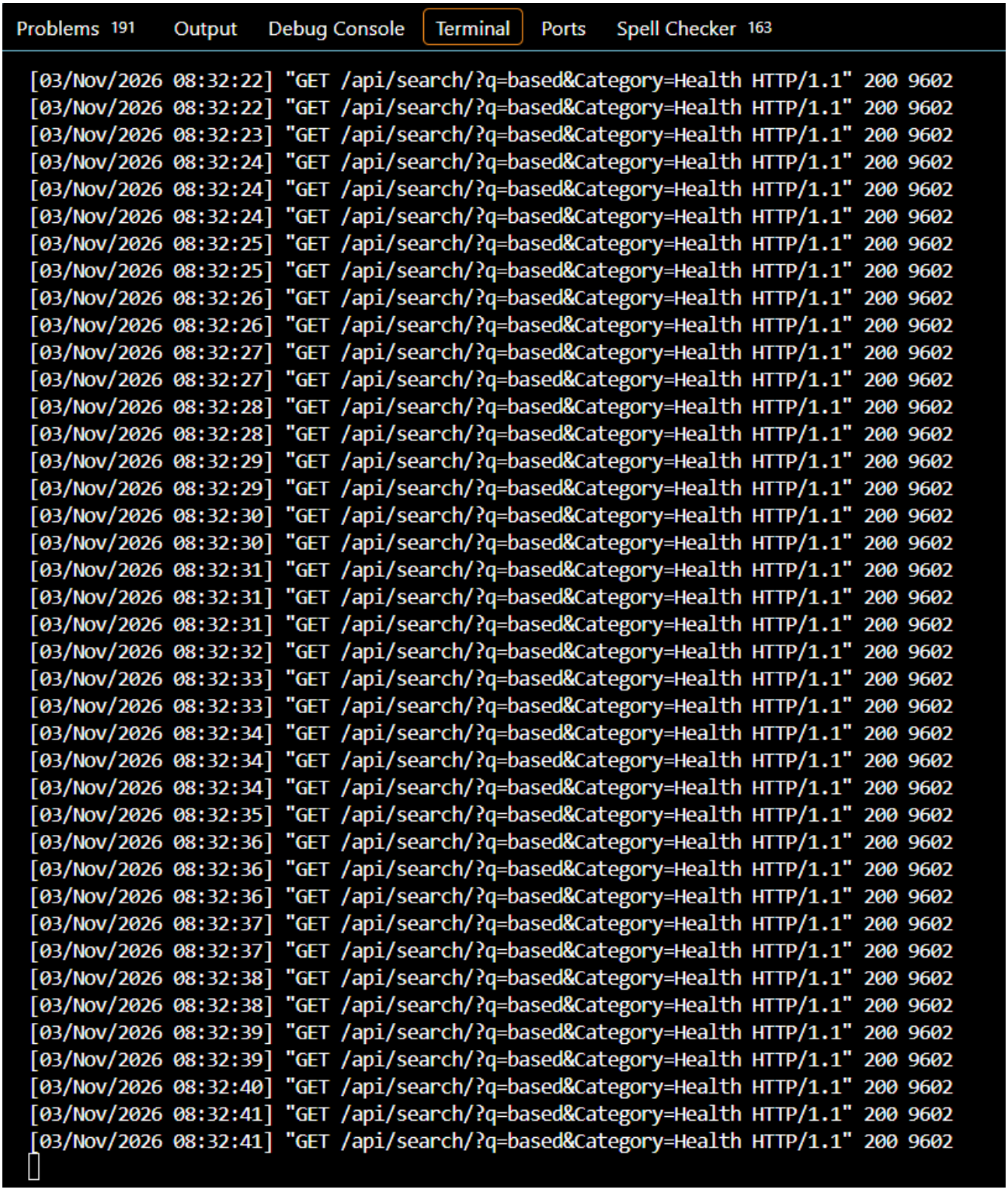


Figure 14: The load tests hitting the same URL from Figure 13.

9. Load Test Results

The tables containing the information from the load tests are below. They are separated into three sections, one for each of the different deployments tested. The tables containing the outputs of each of the load tests per application type will be shown, followed by the Supabase metrics for the database and the metrics for the service used to deploy the application if needed.

The startup and measuring periods for each of the load tests were the same each time. They were kept at the default values of 10 seconds of warmup and 60 seconds of measurement.

Six different queries were tested per each of the deployment methods. They start with a blank query and end with a query that has nearly a thousand copies. The non-blank queries contain a mix of queries with and without a specified category facet. The words chosen for these queries were done so based on testing random words in the search function and seeing how many objects are returned. A debug feature was added to the search page in order to show how many objects were returned in the search. The queries are as follows, followed by the number of objects that appear in the search:

- blank (0)
- based & category=Health (17)
- train (53)
- around (156)
- education & category=Sports (261)
- education (913)

A load test was done 3 times for each of the URLs, the only argument being changed each time was the number of concurrent users. The number of concurrent users in each of these tests was decided to increase from 5 to 50 then furthermore to 150.

For the load tests where the category argument was not used, the Category column for the test will only show an empty string.

Localhost:

Query	Category	Concurrency	Success RPS	P50 Latency (ms)	P99 Latency (ms)	Error Rate
“ ”	“”	5	8.44	590.66	647.52	0
“ ”	“”	50	8.21	4626.81	18804.51	0
“ ”	“”	150	8.01	7859.73	32170.89	0.43
“based”	“Health”	5	1.69	2870.12	3792.7	0
“based”	“Health”	50	2.2	10322.92	30531	0.24
“based”	“Health”	150	2.06	15329.61	32637.36	0.75
“train”	“”	5	0	22231.01	22678.24	1
“train”	“”	50	0.36	22338.46	29297.42	0.83
“train”	“”	150	0.42	30002.92	32588.24	0.91
“around”	“”	5	1.47	3354.09	4169.91	0
“around”	“”	50	1.39	21731.04	30028.61	0.33
“around”	“”	150	0	30009.6	32538.8	1
“education”	“Sports”	5	1.76	2696.6	3785.65	0
“education”	“Sports”	50	1.33	20994.98	31031.71	0.51
“education”	“Sports”	150	0.42	30008.94	32479.02	0.91
“education”	“”	5	1.64	2957.5	4056.8	0
“education”	“”	50	1.19	21679.22	30012.24	0.39
“education”	“”	150	0	30004.93	32597.92	1

Figure 15: Load test results for the localhost



Figure 16: Supabase metrics during the localhost load test

Render:

Query	Category	Concurrency	Success RPS	P50 Latency (ms)	P99 Latency (ms)	Error Rate
“ ”	“”	5	10.6	455.36	898.88	0
“ ”	“”	50	5.06	6235.76	21151.13	0.37
“ ”	“”	150	4.97	3146.45	30341.23	0.87
“based”	“Health”	5	2.29	1735.24	16833.13	0.03
“based”	“Health”	50	0.75	1317.52	18382.22	0.97
“based”	“Health”	150	0.63	3069.78	33483.82	0.98
“train”	“”	5	1.29	2174.42	16755.56	0.08
“train”	“”	50	0.29	1482.42	16487.49	0.99
“train”	“”	150	0.77	2990.61	31964	0.98
“around”	“”	5	1.61	2316.9	16526.06	0.08
“around”	“”	50	0.1	1352.19	11094.22	1
“around”	“”	150	0.44	2921.58	33562.31	0.98
“education”	“Sports”	5	0	638.86	1322.14	1
“education”	“Sports”	50	0.73	1296.76	26398.81	0.97
“education”	“Sports”	150	0.06	3027.61	30122.45	1
“education”	“”	5	0	635.69	1315.32	1
“education”	“”	50	0	1371.84	2270.02	1
“education”	“”	150	0.74	3131.16	30624.82	0.98

Figure 17: Load test results for the Render deployment

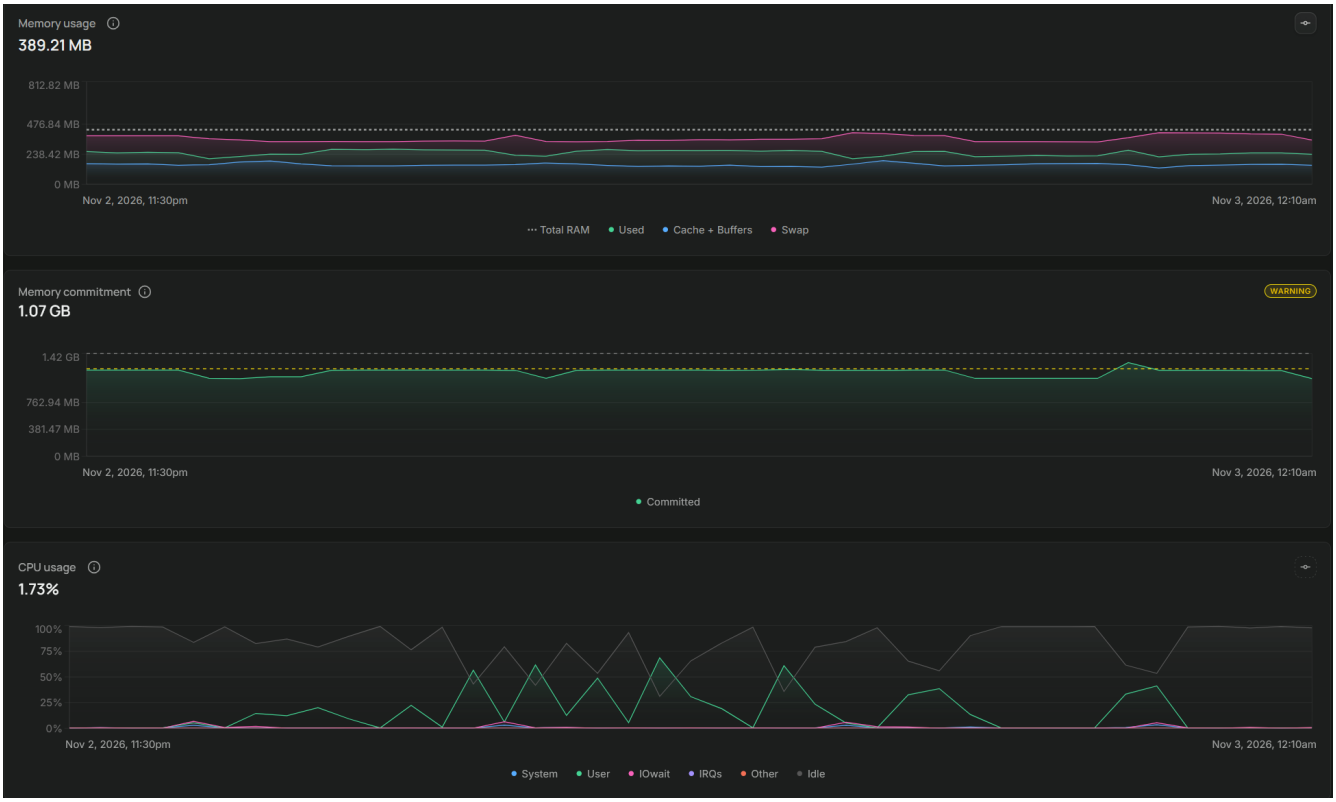


Figure 18: Supabase metrics during the Render load test



Figure 19: Render metrics during the loadtests. There are 7 times in which the instance failed during the tests.

Azure:

Query	Category	Concurrency	Success RPS	P50 Latency (ms)	P99 Latency (ms)	Error Rate
“ ”	“”	5	16.06	267.67	15291.78	0
“ ”	“”	50	50.96	644.5	21054.19	0.01
“ ”	“”	150	59.3	1765.98	21066.87	0.01
“based”	“Health”	5	5.81	850	1498.09	0
“based”	“Health”	50	5.73	8558.32	9668.81	0
“based”	“Health”	150	5.68	25959.99	26820.52	0
“train”	“”	5	2.75	1835.05	2681.33	0
“train”	“”	50	2.72	17813.36	19991.33	0
“train”	“”	150	0.17	30142.54	31151.65	0.95
“around”	“”	5	2.3	2153.48	3170.19	0
“around”	“”	50	2.3	21316.94	22543.03	0
“around”	“”	150	0	30147.81	30168.81	1
“education”	“Sports”	5	4.15	1175.5	2611.53	0
“education”	“Sports”	50	4.4	11169.81	12158.3	0
“education”	“Sports”	150	1.33	30132.46	30234.66	0.71
“education”	“”	5	2.36	2050.35	3043.1	0
“education”	“”	50	2.33	20730.15	22370.23	0
“education”	“”	150	0	30149.58	30178.2	1

Figure 20: Load test results for the Azure deployment

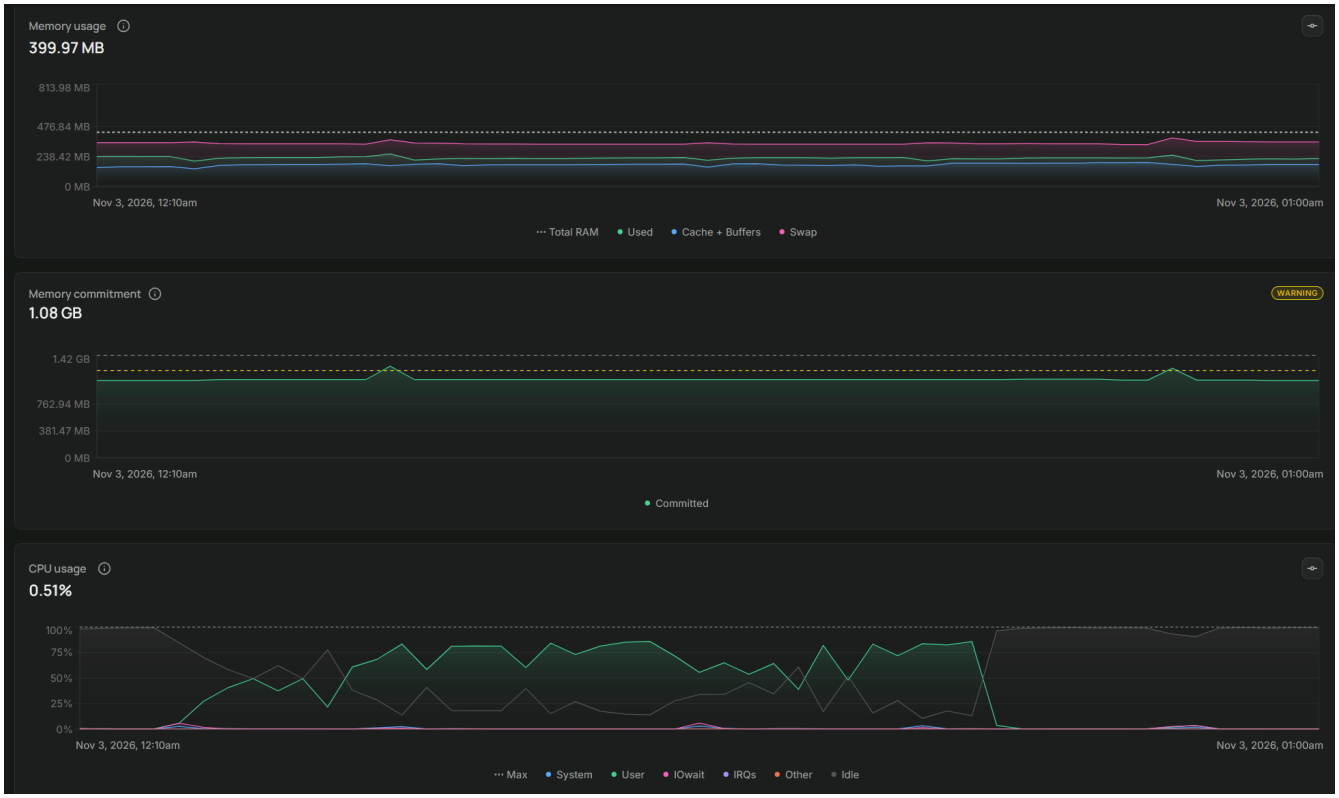


Figure 21: Supabase metrics during the Azre load test

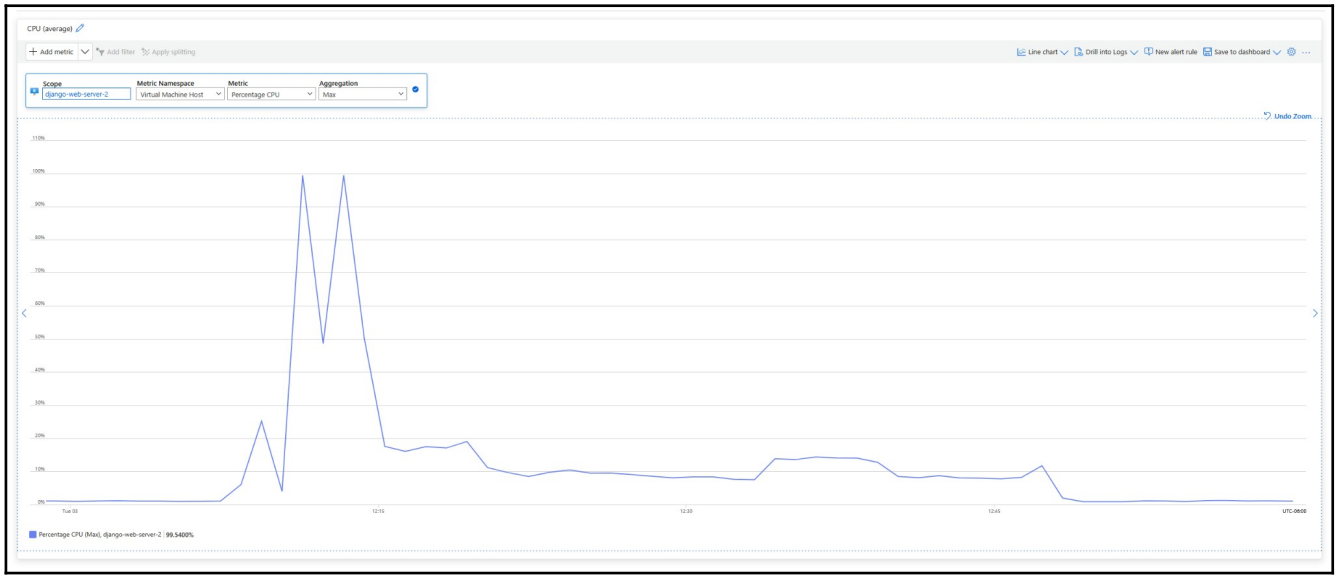


Figure 22: Azure metrics during the load tests.

10. Load Test Findings

Across all of the deployment methods, as both the number of concurrent users and length of returns from the queries increased, there was a decrease in RPS and an increase in response time and error rate.

As stated in Section 5, adding trigram similarity and cosine distance calculations noticeably increased the response time of a search query. Before implementing these, the searches were relatively quick. Adding these calculations took the application a few seconds in order to load queries on the website. Since this was technically when concurrent users was at 1, it meant that adding more concurrent users would make the application slow down even more.

A decrease in RPS as the size of what is somewhat unexpected. The larger a file is, the longer it takes to send that file between hosts, but that usually is made up by the fact that there are more users at once, thus increasing the RPS as they succeed. The main problem found with the load testing, that also contributes to the decreasing RPS, is the extremely higher than average and 99th percentile of the reponse times and the higher error rate as the concurrent users and size of what is returned increases.

On all three different deployments, the average response time grew to dozens of seconds as the number number of users and size increased. At lower concurrency, the time was relatively low compared to the one at high concurrency. For instance, on the “education” query on a local host, the average time at 5 concurrent users was just about under 3 seconds. Compare this to the average time on

the same query at 150 concurrent users, which increased to a bit over 30 seconds. Of course, this query returns the most objects by a long margin compared to the other queries, but even on other queries, this same correlation rings true.

On to application specific findings. Average response times on Render appear to be much lower than the average response times of the other two deployments, but the error rate is significantly increased compared to them as well. Even at lower concurrency levels and lower sizes, the error level is above zero for all of them, and even at concurrency levels of 50 reached a near 100% error rate. One reason for this issue could be found in the fact that multiple times during the load testing on Render, the instance would fail. During the load tests, the available memory and CPU would spike up and go to near one hundred percent. The amount of data needed to be transferred by the concurrent users was clearly too much for the render. All of the misses seen via Render (seen in the m5/results folder load tests that start with “R” for Render) are 502 errors for a bad gateway.

The localhost had a better error rate than the Render deployed application, but the response times were way longer. There is a clear correlation between the number of concurrent users and the error rate. For nearly all of the tests with only 5 concurrent users, there was an error rate of zero. All of the misses for the localhost have a similar structure to this:

```
2039.6814999985509,0,URLError: <urlopen error [WinError 10061] No connection could
        be made because the target machine actively refused it>
```

It appears that a similar issue with Render was happening.

Azure had the lowest error rate of all the deployment types, but it had similar response times to the localhost. It was the only deployment type where the number of tests with an error rate of zero outmatched the ones that had an error rate above zero. It appears the bottleneck for Azure is the number of concurrent users. For all queries, the error rate for both 5 and 50 concurrent users was near zero, and for 4 queries with the most size, the error rate increases significantly as the increased. The lowest of these specific error rates is just above 71%. Unlike the other two methods, there did not appear to be any time that Azure went down. All of the misses were due to timeout errors.

In terms of deployment being consistent, Azure is better than Render or localhost since it did not crash during the load testing and only returned misses.

12. Conclusion

The load tests for this milestone show that there needs to be improvements to our application as the number of concurrent users and the size of the data sent is increased. There is a clear decrease in performance as these two variables increase. Higher response times and error rates correlate with an increase in concurrent users and the size of a query. Even after the indexing done in Milestone 4, the response times are much longer than they should be.

There are many ways that these attributes can be lowered. The similarity aspect seen in pgvector could be tweaked to return less objects. Minimum text similarity could be increased even more to eliminate unwanted objects. The trigrams could also be used in more attibutes. This would increase the number of trigrams inside of an event, making it harder to achieve similarity with the query. The same objective can be achieved by decreasing the cosine distance required for an object to be considered into the query. As stated, the addition of cosign distance and trigram similarity did add a lot of time to the responses compared to before they were added.

While the performance is not desired, the fact that the search does perform successfully is good.

The main performance takeaway from this milestone is that added computation increases the response time and increases the error rate. The longer it takes for data to be transferred, the higher the likelihood that one part will return bad or make the user time out.